## OTW_ Wargames

# OVERTHEWIRE BANDIT WARGAME

**PART 1**

***Practical Linux & Privilege Escalation Fundamentals for Junior Pentesters***

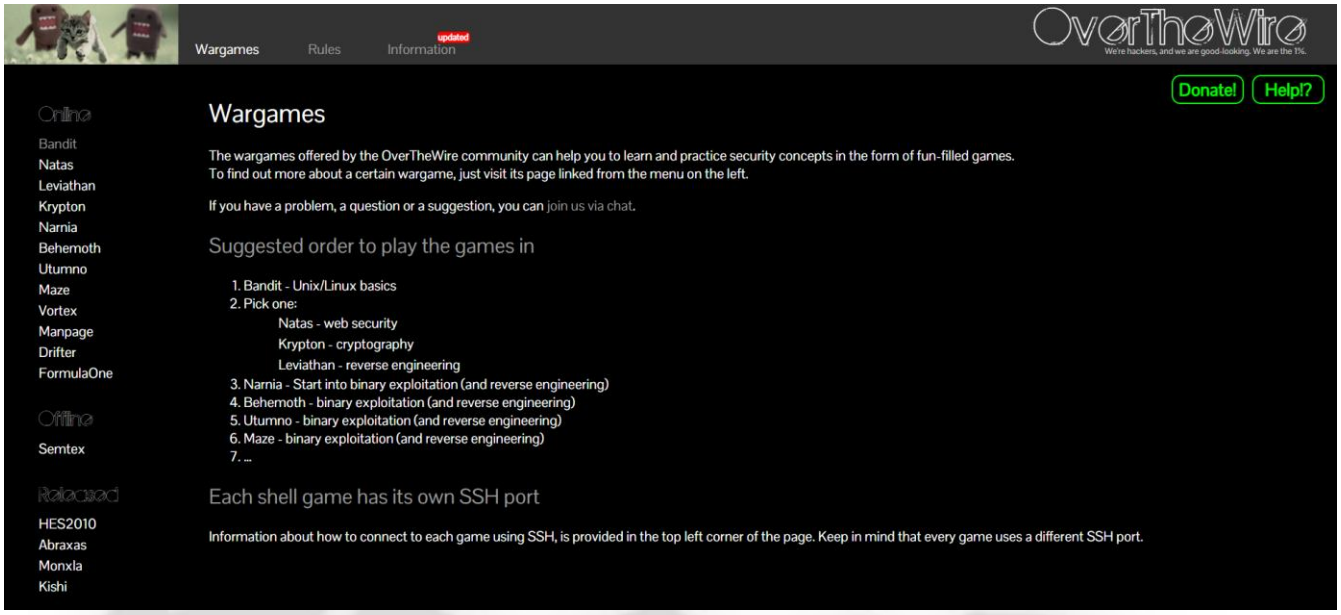

**Prepared by**

**Ahmed Eldkrory (EDGAR)**

**Junior Penetration Tester**

**ahmed.a.eldkrory@gmail.com**
**www.linkedin.com/in/ahmed-eldkrory**

# Overview




This document demonstrates my understanding of fundamental Linux and command-line concepts from a security perspective, based on the **OverTheWire Bandit Wargame**. It summarizes key techniques, methodologies, and learning outcomes developed while solving the initial levels of the challenge.

The focus of this write-up is on **technical reasoning, problem-solving approach, and practical skill development**, rather than simply listing solutions or commands. Emphasis is placed on understanding how each task reflects real-world scenarios encountered in penetration testing and system analysis.

This report specifically covers **Levels 0 → 10 (first 10 levels)** of the Bandit Wargame. Due to the progressive nature of the challenge, the full walkthrough will be divided into **three separate reports**, each addressing a different range of levels to maintain clarity and depth of explanation.

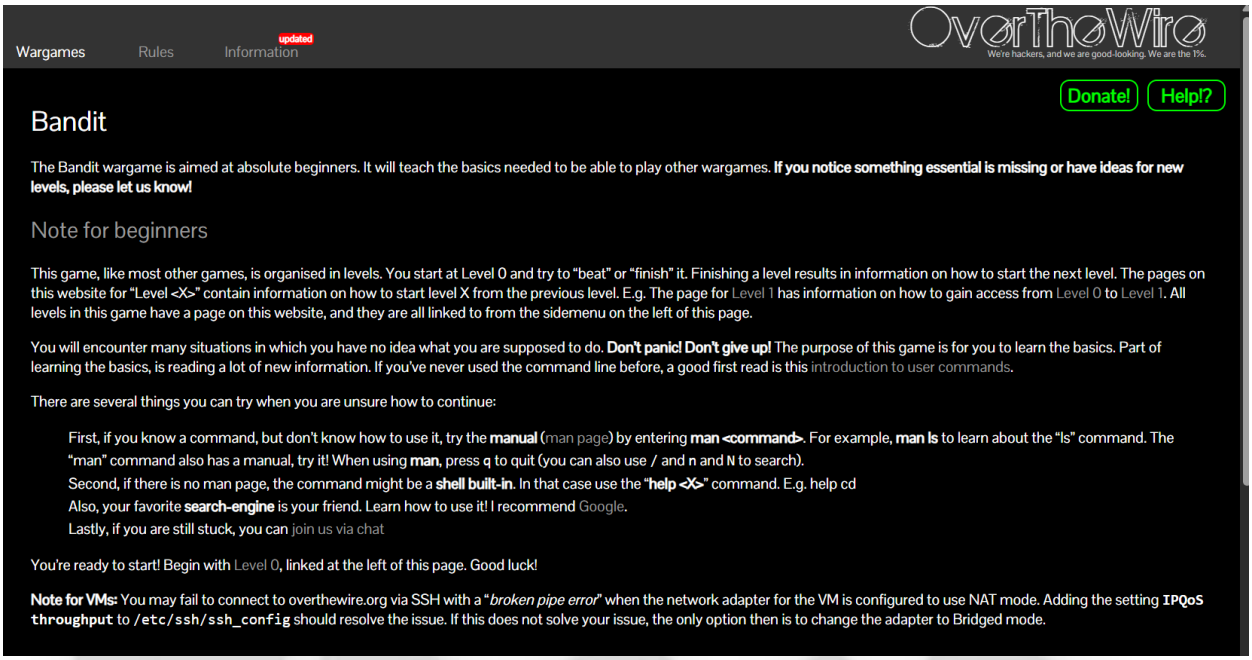
Full Write-up:

https://medium.com/@ahmed.a.eldkrory/overthewire-bandit-levels-0-10-from-linux-fundamentals-to-data-extraction-a9ac037d8e38

Wargame Link:
https://overthewire.org/wargames/bandit/

---

# Bandit Wargame



Wargames Rules Information updated

OverTheWire
We're hackers, and we are good-looking. We are the 1%.

Donate! Help!?

## Bandit

The Bandit wargame is aimed at absolute beginners. It will teach the basics needed to be able to play other wargames. **If you notice something essential is missing or have ideas for new levels, please let us know!**

### Note for beginners

This game, like most other games, is organised in levels. You start at Level 0 and try to "beat" or "finish" it. Finishing a level results in information on how to start the next level. The pages on this website for "Level <X>" contain information on how to start level X from the previous level. E.g. The page for Level 1 has information on how to gain access from Level 0 to Level 1. All levels in this game have a page on this website, and they are all linked to from the sidemenu on the left of this page.

You will encounter many situations in which you have no idea what you are supposed to do. **Don't panic! Don't give up!** The purpose of this game is for you to learn the basics. Part of learning the basics, is reading a lot of new information. If you've never used the command line before, a good first read is this introduction to user commands.

There are several things you can try when you are unsure how to continue:

First, if you know a command, but don't know how to use it, try the **manual** (man page) by entering **man <command>**. For example, **man ls** to learn about the "ls" command. The "man" command also has a manual, try it! When using **man**, press **q** to quit (you can also use / and **n** and **N** to search).
Second, if there is no man page, the command might be a **shell built-in**. In that case use the "**help <X>**" command. E.g. help cd
Also, your favorite **search-engine** is your friend. Learn how to use it! I recommend Google.
Lastly, if you are still stuck, you can join us via chat

You're ready to start! Begin with Level 0, linked at the left of this page. Good luck!

**Note for VMs:** You may fail to connect to overthewire.org via SSH with a "*broken pipe error*" when the network adapter for the VM is configured to use NAT mode. Adding the setting `IPQoS throughput` to `/etc/ssh/ssh_config` should resolve the issue. If this does not solve your issue, the only option then is to change the adapter to Bridged mode.

The goal of the Bandit Wargame is to introduce beginners to essential Linux skills required in cybersecurity, particularly in penetration testing and system exploitation.

The challenge is structured as a sequence of levels, where each level requires the user to retrieve a password that grants access to the next level. These passwords are typically hidden within files, directories, or system outputs, requiring the use of Linux commands and logical reasoning to uncover them.

Unlike theoretical learning, Bandit emphasizes **hands-on interaction with a real Linux environment**, encouraging users to think like attackers while developing practical command-line proficiency.

For example, a user may need to:

- Navigate complex directory structures
- Identify hidden files
- Read file contents with restricted permissions
- Process and filter command output

Each level builds upon the previous one, reinforcing core skills and introducing new concepts gradually.

# Why Bandit Matters in Cybersecurity

Mastering the Linux command line is a foundational requirement in cybersecurity. The Bandit Wargame directly supports this by simulating realistic scenarios that security professionals encounter.

From a penetration testing perspective, these skills are critical for:

- Navigating compromised systems efficiently
- Locating sensitive files and credentials
- Understanding file permissions and access control
- Automating tasks using command-line tools
- Interacting with remote systems via SSH

Additionally, Bandit helps develop a **problem-solving mindset**, which is essential for:

- Privilege escalation
- Post-exploitation activities
- Capture The Flag (CTF) challenges
- Real-world security assessments

By understanding how attackers think and operate at a low level, defenders can better secure systems against similar techniques.

---

# Core Concept: Linux Command-Line Fundamentals

At its core, the Bandit Wargame focuses on mastering the Linux command-line interface (CLI), which is a primary tool for both attackers and defenders.

Key concepts covered include:

- **File System Navigation**
  Understanding how to move through directories and locate files using commands such as `cd`, `ls`, and `pwd`.
- **File Inspection and Manipulation**
  Reading and analyzing file contents using tools like `cat`, `less`, `head`, and `tail`.
- **Permissions and Access Control**
  Interpreting file permissions and understanding how they affect accessibility.
- **Data Processing**
  Filtering and extracting useful information using commands such as `grep`, `sort`, and `uniq`.
- **Remote Access**
  Connecting to systems securely using SSH and managing authentication credentials.

A useful way to think about the command line is as a **direct interface to the operating system**, where every action is explicit and controlled.

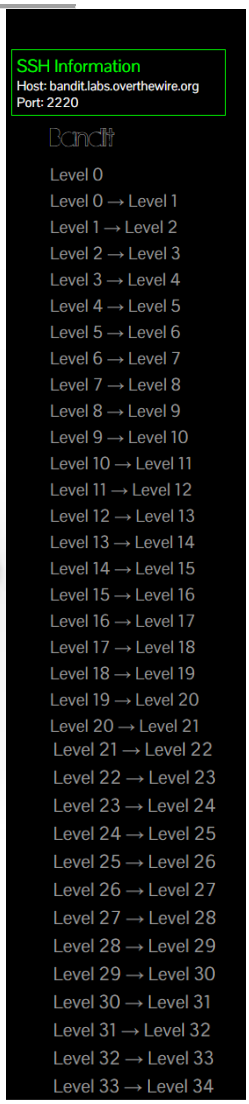
Unlike graphical interfaces, the CLI provides:

- Greater precision
- More control over system operations
- The ability to automate complex tasks

These properties make it an essential tool in cybersecurity operations.

---

# Room Structure




The Bandit Wargame is structured as a sequence of progressively challenging levels, each designed to introduce a specific concept or skill.

In this report, the focus is on:

### Levels 0 → 10 (Foundational Stage)

This stage emphasizes:

- Basic Linux commands
- File handling techniques
- Understanding hidden and non-standard file names
- Introductory data extraction methods

Each level builds incrementally, ensuring a solid foundation before moving to more

advanced topics in later stages.

**Operational Tip: Level Transitions**

**Methodology** To progress efficiently, you must close the current session and

re-authenticate as the next user. Avoid nesting SSH sessions,as this creates confusion and connection instability.

**Commands**

`exit` — Terminate the current session and return to your local terminal.

`ssh bandit[N]@bandit.labs.overthewire.org -p 2220` — Connect to the next level (where [N] is the new level number).

# Bandit Level 0

Donate! Help!?

## Bandit Level 0

### Level Goal

The goal of this level is for you to log into the game using SSH. The host to which you need to connect is **bandit.labs.overthewire.org**, on port 2220. The username is **bandit0** and the password is **bandit0**. Once logged in, go to the Level 1 page to find out how to beat Level 1.

### Goal

Establish an initial remote connection to the Bandit wargame server via SSH.

### Technique

Remote access via Secure Shell (SSH) using a specified username and a non-standard port to bypass basic network noise.

### Commands

- `ssh bandit0@bandit.labs.overthewire.org -p 2220` – Connects to the remote server using the username bandit0 on port 2220.

### Result

Successfully authenticated and gained access to the `bandit0` shell environment.

### Execution & Documentation

```
┌──(edgar㉿kali)-[~]
└─$ ssh bandit0@bandit.labs.overthewire.org -p 2220

                         _                 _ _ _
                        | |__   __ _ _ __   __| (_) |_
                        | '_ \ / _` | '_ \ / _` | | __|
                        | |_) | (_| | | | | (_| | | |_
                        |_.__/ \__,_|_| |_|\__,_|_|\__|


                      This is an OverTheWire game server.
             More information on http://www.overthewire.org/wargames

backend: gibson-0
bandit0@bandit.labs.overthewire.org's password:

      ,----..            ,----,         .---.
     /   /   \        ,/   .`|        /. ./|
    /   .     :      ,`   .'  :     .--'.  ' ;
   .   /   ;.  \   ;    ;     /    /__./ \ : |
  .   ;   /  ` ; .'___,/    ,' .--'.   '   \' .
  ;   |  ; \ ; | |    :    | /___/ \ |    ' '
  |   :  | ; | ' ;    |.';  ; ;    \  \;      :
  .   |  ' ' ' : `----'  |  |  \   ;  `      |
  '   ;  \; /  |      '   :  ;   .   \    .\  ;
   \   \  ',  /       |   |  '    \   \   ' \ |
    ;   :    /        '   :  |     :    '  |--"
     \   \ .'          ;   |.'       \   \ ;
  www. `---` ver      '---' he        '---" ire.org


Welcome to OverTheWire!
```

# Bandit Level 0 → 1

Donate! Help!?

Bandit Level 0 → Level 1

Level Goal

The password for the next level is stored in a file called **readme** located in the home directory. Use this password to log into bandit1 using SSH. Whenever you find a password for a level, use SSH (on port 2220) to log into that level and continue the game.

### Goal

Locate and read the `readme` file in the home directory to find the next level's password.

### Technique

Basic system reconnaissance and data extraction from standard text files.

### Commands

- `ls` — Lists the contents of the current directory to identify available files.
- `cat readme` — Displays the text content of the readme file to the terminal.

### Result

Password Found: `ZjLjTmm6FvvyRnrb2rfNWOZOTa6ip5If`

### Execution & Documentation

```
bandit0@bandit:~$ ls
readme
bandit0@bandit:~$ cat readme
Congratulations on your first steps into the bandit game!!
Please make sure you have read the rules at https://overthewire.org/rules/
If you are following a course, workshop, walkthrough or other educational activity,
please inform the instructor about the rules as well and encourage them to
contribute to the OverTheWire community so we can keep these games free!

The password you are looking for is: ZjLjTmM6FvvyRnrb2rfNWOZOTa6ip5If

bandit0@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

# Bandit Level 1 → 2

Donate! Help!?

## Bandit Level 1 → Level 2

### Level Goal

The password for the next level is stored in a file called - located in the home directory

### Commands you may need to solve this level

ls , cd , cat , file , du , find

**Goal**

Retrieve the password from a file named – located in the home directory.

**Technique**

Managing special character filenames by using relative path referencing (`./`) to prevent the shell from misinterpreting the filename as a command argument.

**Commands**

- `ls` – Confirms the existence of the file named - in the current directory.
- `cat ./-` – Forces the system to read the file by specifying its path, avoiding argument errors.

**Result**

Password Found: `263JGJPfgU6LtdEvgfWU1XP5yac29mFx`

**Execution & Documentation**

```
bandit1@bandit:~$ ls
-
bandit1@bandit:~$ cat ./-
263JGJPfgU6LtdEvgfWU1XP5yac29mFx
bandit1@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

# Bandit Level 2 → 3

Donate! Help!?

## Bandit Level 2 → Level 3

### Level Goal

The password for the next level is stored in a file called `--spaces in this filename--` located in the home directory

**Goal**

Retrieve the password from a file named `--spaces in this filename--` located in the home directory.

**Technique**

Handling filenames with spaces. In Linux, spaces act as delimiters between commands and arguments. To read such files, you must either use **backslash escaping** for every space or wrap the entire filename in **quotes**.

**Commands**

- `ls -la` — Lists all files, including hidden ones, with detailed permissions and attributes.
- `cat ./\--spaces\ in\ this\ filename--` — Uses backslashes to escape spaces and a relative path to handle the leading dashes.

**Result**

Password Found: `MNk8KNH3Usiio41PRUEoDFPqfxLPlSmx`

**Execution & Documentation**

```
bandit2@bandit:~$ ls -la
total 24
drwxr-xr-x   2 root    root    4096 Oct 14 09:26 .
drwxr-xr-x 150 root    root    4096 Oct 14 09:29 ..
-rw-r--r--   1 root    root     220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root    root    3851 Oct 14 09:19 .bashrc
-rw-r--r--   1 root    root     807 Mar 31  2024 .profile
-rw-r-----   1 bandit3 bandit2   33 Oct 14 09:26 --spaces in this filename--
bandit2@bandit:~$ cat ./--spaces\ in\ this\ filename--
MNk8KNH3Usiio41PRUEoDFPqfxLPlSmx
bandit2@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.

┌──(edgar㉿kali)-[~]
└─$
```

# Bandit Level 3 → 4

Donate! Help!?

## Bandit Level 3 → Level 4

### Level Goal

The password for the next level is stored in a hidden file in the **inhere** directory.

**Goal**

Find a hidden file within the `inhere` directory to retrieve the next password.

**Technique**

Identifying hidden files. In Linux, any file starting with a dot (`.`) is hidden and does not appear with a standard `ls` command. Professional reconnaissance requires using specific flags to reveal these "dotfiles."

**Commands**

- `cd inhere/` — Changes the current working directory to the target folder.
- `ls -la` — Uses the `-a` (all) flag to reveal hidden files within the directory.
- `cat ...Hiding-From-You` — Displays the contents of the identified hidden file.

**Result**

Password Found: `2WmrDFRmJIq3IPxneAaMGhap0pFhF3NJ`

**Execution & Documentation**

```
bandit3@bandit:~$ ls
inhere
bandit3@bandit:~$ cd inhere/
bandit3@bandit:~/inhere$ ls -la
total 12
drwxr-xr-x 2 root    root    4096 Oct 14 09:26 .
drwxr-xr-x 3 root    root    4096 Oct 14 09:26 ..
-rw-r----- 1 bandit4 bandit3   33 Oct 14 09:26 ...Hiding-From-You
bandit3@bandit:~/inhere$ cat ...Hiding-From-You
2WmrDFRmJIq3IPxneAaMGhap0pFhF3NJ
bandit3@bandit:~/inhere$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

**Bandit Level 4 → 5**

Donate! Help!?

## Bandit Level 4 → Level 5

### Level Goal

The password for the next level is stored in the only human-readable file in the **inhere** directory. Tip: if your terminal is messed up, try the "reset" command.

**Goal**

Identify and read the only human-readable file within the inhere directory.

**Technique**

Determining file types. In Linux, files do not rely on extensions (like .txt) to define their content. Security professionals use the file command to inspect file headers and determine if a file contains data, executable code, or ASCII text.

**Commands**

- `cd inhere/` — Navigates into the target directory.
- `file ./*` — Inspects every file in the directory to identify its format.
- `cat ./-file07` — Reads the file identified as "ASCII text" using a relative path to handle the leading dash.

**Result**

Password Found: `4oQYVPkxZOOE0O5pTW81FB8j8lxXGUQw`

**Execution & Documentation**

```
bandit4@bandit:~$ ls -la
total 24
drwxr-xr-x   3 root root 4096 Oct 14 09:26 .
drwxr-xr-x 150 root root 4096 Oct 14 09:29 ..
-rw-r--r--   1 root root  220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root root 3851 Oct 14 09:19 .bashrc
drwxr-xr-x   2 root root 4096 Oct 14 09:26 inhere
-rw-r--r--   1 root root  807 Mar 31  2024 .profile
```

```
bandit4@bandit:~$ cd inhere/
bandit4@bandit:~/inhere$ ls -la
total 48
drwxr-xr-x 2 root    root    4096 Oct 14 09:26 .
drwxr-xr-x 3 root    root    4096 Oct 14 09:26 ..
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file00
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file01
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file02
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file03
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file04
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file05
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file06
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file07
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file08
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file09
bandit4@bandit:~/inhere$ file -- *
-file00: data
-file01: data
-file02: data
-file03: data
-file04: data
-file05: data
-file06: data
-file07: ASCII text
-file08: data
-file09: data
bandit4@bandit:~/inhere$ cat -- -file07
4oQYVPkxZOOEOO5pTW81FB8j8lxXGUQw
bandit4@bandit:~/inhere$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

---

**Bandit Level 5 → 6**

Donate! Help!?

Bandit Level 5 → Level 6

Level Goal

The password for the next level is stored in a file somewhere under the **inhere** directory and has all of the following properties:

human-readable
1033 bytes in size
not executable

**Goal**

Find a file hidden deep within the inhere directory structure that matches specific properties: human-readable, 1033 bytes in size, and not executable.

**Technique**

Advanced file searching. When dealing with large directory trees, the find command is essential for filtering files based on metadata like size and permissions, which is a critical skill for locating sensitive configuration files or "flags."

## Commands

- `find . -type f -size 1033c ! -executable` — Searches the current directory for a file (-type f) that is exactly 1033 bytes (-size 1033c) and is not executable (! -executable).
- `cat ./maybehere07/.file2` — Reads the specific file returned by the search query.

## Result

Password Found: `HWasnPhtq9AVKe0dmk45nxy20cvUa6EG`

## Execution & Documentation

```
bandit5@bandit:~$ ls -la
total 24
drwxr-xr-x   3 root root    4096 Oct 14 09:26 .
drwxr-xr-x 150 root root    4096 Oct 14 09:29 ..
-rw-r--r--   1 root root     220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root root    3851 Oct 14 09:19 .bashrc
drwxr-x---  22 root bandit5 4096 Oct 14 09:26 inhere
-rw-r--r--   1 root root     807 Mar 31  2024 .profile
bandit5@bandit:~$ cd inhere/
bandit5@bandit:~/inhere$ ls -la
total 88
drwxr-x--- 22 root bandit5 4096 Oct 14 09:26 .
drwxr-xr-x  3 root root    4096 Oct 14 09:26 ..
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere00
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere01
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere02
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere03
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere04
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere05
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere06
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere07
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere08
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere09
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere10
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere11
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere12
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere13
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere14
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere15
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere16
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere17
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere18
drwxr-x---  2 root bandit5 4096 Oct 14 09:26 maybehere19
bandit5@bandit:~/inhere$ find . -type f -size 1033c ! -executable
./maybehere07/.file2
bandit5@bandit:~/inhere$ file ./maybehere07/.file2
./maybehere07/.file2: ASCII text, with very long lines (1000)
bandit5@bandit:~/inhere$ cat ./maybehere07/.file2
HWasnPhtq9AVKe0dmk45nxy20cvUa6EG

bandit5@bandit:~/inhere$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

**Bandit Level 6 → 7**

Donate! Help!?

Bandit Level 6 → Level 7

Level Goal

The password for the next level is stored **somewhere on the server** and has all of the following properties:

owned by user bandit7
owned by group bandit6
33 bytes in size

**Goal**

Locate a specific 33-byte file owned by user bandit7 and group bandit6 hidden somewhere on the server.

**Technique**

System-wide discovery based on ownership. This level simulates a scenario where a tester must find files belonging to a target user across the entire filesystem. Using 2>/dev/null is critical to filter out "Permission denied" errors, allowing the relevant result to stand out.

**Commands**

- `find / -type f -user bandit7 -group bandit6 -size 33c 2>/dev/null` — Searches the root directory (/) for a file matching specific owner, group, and size criteria, while suppressing error messages.
- `cat /var/lib/dpkg/info/bandit7.password` — Reads the password file located by the search.

**Result**

Password Found: `morbNTDKSW6jIlUc0ymOdMaLnOlFVAAj`

**Execution & Documentation**

```
bandit6@bandit:~$ find / -type f -user bandit7 -group bandit6 -size 33c 2>/dev/null
/var/lib/dpkg/info/bandit7.password
bandit6@bandit:~$ cat /var/lib/dpkg/info/bandit7.password
morbNTDkSW6jIlUc0ymOdMaLnOlFVAaj
bandit6@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

**Bandit Level 7 → 8**

Donate! Help!?

# Bandit Level 7 → Level 8

## Level Goal

The password for the next level is stored in the file **data.txt** next to the word **millionth**

**Goal**

Extract the password from a large file named data.txt where it is stored next to the word "millionth".

**Technique**

Pattern matching in large datasets. When a file is too large to read manually, the grep utility is used to search for specific strings. This is a fundamental skill for log analysis and credential harvesting.

**Commands**

- `ls -la` — Checks the file size (approx. 4MB), confirming manual reading is inefficient.
- `grep millionth data.txt` — Searches for the keyword "millionth" and prints the corresponding line.

**Result**

Password Found: `dfwvzFQi4mU0wfNbFOe9RoWskMLg7eEc`

**Execution & Documentation**

```
bandit7@bandit:~$ ls -la
total 4108
drwxr-xr-x   2 root    root       4096 Oct 14 09:26 .
drwxr-xr-x 150 root    root       4096 Oct 14 09:29 ..
-rw-r--r--   1 root    root        220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root    root       3851 Oct 14 09:19 .bashrc
-rw-r-----   1 bandit8 bandit7 4184396 Oct 14 09:26 data.txt
-rw-r--r--   1 root    root        807 Mar 31  2024 .profile
bandit7@bandit:~$ grep millionth data.txt
millionth       dfwvzFQi4mU0wfNbFOe9RoWskMLg7eEc
bandit7@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

**Bandit Level 8 → 9**

Donate! Help!?

# Bandit Level 8 → Level 9

## Level Goal

The password for the next level is stored in the file **data.txt** and is the only line of text that occurs only once

**Goal**

Identify the only unique line of text within data.txt that occurs exactly once.

**Technique**

Data deduplication and filtering. To find a unique line, the data must first be ordered using sort because uniq only compares adjacent lines. This "pipelining" of commands is a core concept in Linux data processing.

**Commands**

- `sort data.txt | uniq -u` — Sorts the file alphabetically and then filters for lines that appear only once (-u).

**Result**

Password Found: `4CKMh1JI91bUIZZPXDqGanal4xvAg0JM`

**Execution & Documentation**

```
bandit8@bandit:~$ ls -la
total 56
drwxr-xr-x   2 root    root     4096 Oct 14 09:26 .
drwxr-xr-x 150 root    root     4096 Oct 14 09:29 ..
-rw-r--r--   1 root    root      220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root    root     3851 Oct 14 09:19 .bashrc
-rw-r-----   1 bandit9 bandit8 33033 Oct 14 09:26 data.txt
-rw-r--r--   1 root    root      807 Mar 31  2024 .profile
bandit8@bandit:~$ sort data.txt | uniq -u
4CKMh1JI91bUIZZPXDqGanal4xvAg0JM
bandit8@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

**Bandit Level 9 → 10**

Donate! Help!?

# Bandit Level 9 → Level 10

## Level Goal

The password for the next level is stored in the file **data.txt** in one of the few human-readable strings, preceded by several '=' characters.

**Goal**

Retrieve a human-readable string from a binary file (data.txt) preceded by several = characters.

**Technique**

Binary data inspection. Standard text editors and cat often fail to display binary data correctly. The strings command extracts printable character sequences from binary files, which is essential for reverse engineering and discovering hidden flags in compiled programs.

**Commands**

- `strings data.txt | grep "==="` — Extracts all readable strings and filters for those containing multiple equals signs.

**Result**

Password Found: `FGUW5ilLVJrxX9kMYMmlN4MgbpfMiqey`

**Execution & Documentation**

```
bandit9@bandit:~$ ls -la
total 40
drwxr-xr-x   2 root     root     4096 Oct 14 09:25 .
drwxr-xr-x 150 root     root     4096 Oct 14 09:29 ..
-rw-r--r--   1 root     root      220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root     root     3851 Oct 14 09:19 .bashrc
-rw-r-----   1 bandit10 bandit9 19382 Oct 14 09:25 data.txt
-rw-r--r--   1 root     root      807 Mar 31  2024 .profile
bandit9@bandit:~$ strings data.txt | grep "==="
========== the
========== password
E========== is
5========== FGUW5ilLVJrxX9kMYMmlN4MgbpfMiqey
bandit9@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

## Conclusion

This segment of the OverTheWire Bandit wargame provided foundational hands-on experience with Linux system administration and command-line navigation from a security perspective. Through these initial 11 levels, I developed a systematic approach to exploring remote environments and interacting with various file structures and data formats.

During the early levels, the focus was on establishing secure remote access via SSH and mastering basic directory reconnaissance. This demonstrated the importance of understanding how to navigate a target system efficiently to locate "low-hanging fruit," such as clear-text credentials and sensitive documentation left in home directories.

As the challenges progressed, the difficulty shifted toward handling specialized file scenarios, such as dealing with hidden "dotfiles," escaping special characters, and managing filenames that conflict with command-line arguments. These levels reinforced the technical precision required by a penetration tester to avoid unintended shell execution or command errors during an assessment.

The final levels in this range introduced more advanced data processing techniques, including system-wide file discovery using metadata (size, ownership, and permissions) and the use of pipelines to filter large datasets. Utilizing tools like grep, sort, uniq, and strings highlighted how security professionals extract actionable intelligence from binary data and high-volume logs.

Overall, this first stage of the wargame strengthened my understanding of:

- **Secure Remote Access:** Interacting with SSH on non-standard ports.
- **System Reconnaissance:** Identifying hidden files and analyzing file types.
- **Data Processing:** Using command-line piping to filter and deduplicate information.
- **Binary Analysis:** Extracting human-readable strings from non-text files.

These skills are the fundamental building blocks for any security professional, providing the baseline competency required for privilege escalation, post-exploitation analysis, and efficient system auditing.

---

## Final Notes

This documentation is intentionally high-level and ethical. It demonstrates learning outcomes and professional reasoning without exposing lab content, answers, or sensitive details.

---